

COMPUTACIÓ PARAL·LELA I MASSIVA - 1.1

Paral·lelització amb OpenMP de K-means

Nom: Jordà Coca Rodríguez

Curs: 2025 - 2026

Índex

1. Enunciat
2. Anàlisi de les dependències
3. Proposta de paral·lelització
4. Codi de la proposta
5. Execució i metodologia de mesura
6. Resultats
7. Validació de la sortida
8. Conclusions

1. Enunciat

L'objectiu d'aquesta pràctica és paral·lelitzar amb OpenMP un algorisme de càlcul d'agrupacions K-means sobre 600.000 elements i 200 grups.

El temps d'execució seqüencial de referència amb optimització -O3 és de 17,3 segons a Orca i 8,8 segons a Teen segons l'enunciat. A les execucions finals també s'ha mesurat la versió paral·lela amb 1 thread per usar-la com a base experimental de comparació dins de cada màquina.

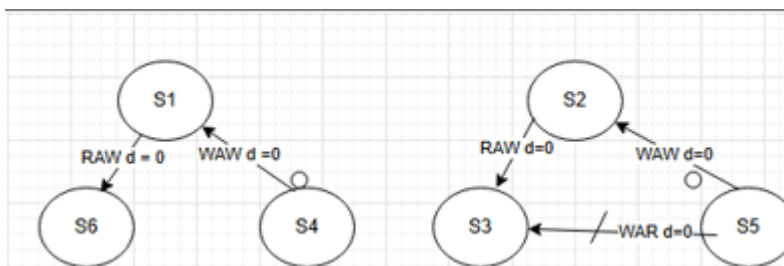
La versió paral·lela ha de donar el mateix resultat que la versió seqüencial, i cal comprovar els valors finals de sortida. També cal executar el codi forçant la creació de threads en potències de 2: de 2 fins a 32 a Teen i de 2 fins a 128 a Orca.

La mitjana harmònica mínima de speedup que s'ha d'assolir és 4 a Teen i 6 a Orca.

2. Anàlisi de les dependències

En aquest apartat analitzo les dependències dels bucles principals del programa, especialment els bucles de la funció kmean. La funció qs no s'ha paral·lelitzat, ja que només ordena els 200 representants finals i el seu cost és molt petit comparat amb el càlcul principal.

2.1. Bucle 1: assignació de clusters



```
for (i=0; i<fN; i++)
{
    min = 0;           S1
    dif = abs(fV[i] - fR[0]); S2
    for (j=1; j<fK; j++){
        if (abs(fV[i] - fR[j]) < dif) S3
        {
            min = j;           S4
            dif = abs(fV[i] - fR[j]); S5
        }
        fD[i] = min;           S6
    }
}
```

El primer bucle és la part més important del codi. En aquest bucle, per a cada element de V, es busca el centroide R[j] més proper i es guarda el cluster assignat a fD[i]. El seu cost és aproximadament $O(N \cdot G)$, on $N = 600.000$ i $G = 200$. Per això és el coll de botella principal de l'algorisme.

Respecte les dependències, cada iteració del bucle exterior treballa sobre un valor diferent de i. Les lectures de fV i fR són compartides però només de lectura, i l'escriptura es fa sempre sobre una posició diferent de fD[i]. Per tant, no hi ha dependències transportades pel bucle entre iteracions diferents del bucle exterior.

Les variables auxiliars que s'usen dins del bucle, com j, min i best_dif, han de ser privades per a cada thread. En la versió final, min i best_dif estan declarades dins del cos del bucle, de manera que ja són locals a cada iteració, i j s'indica com a private a la directiva OpenMP.

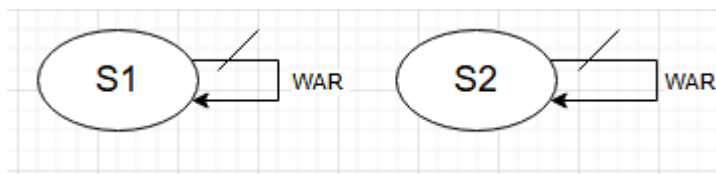
2.2. Bucle 2: inicialització dels acumuladors

```
for(i=0;i<fK;i++)  
    fS[i] = 0;  
    fA[i] = 0;
```

Aquest bucle inicialitza els arrays fS i fA. fS conté la suma dels valors dels elements de cada cluster, mentre que fA conté el nombre d'elements assignats a cada cluster.

No hi ha dependències entre iteracions, ja que cada iteració inicialitza una posició diferent dels arrays. Tot i això, en la versió final s'ha deixat seqüencial perquè només té 200 iteracions i el cost de crear i sincronitzar threads pot ser superior al benefici obtingut.

2.3. Bucle 3: acumulació dels valors per cluster



```
for(i=0;i<fN;i++)  
{  
    fS[fD[i]] += fV[i];    S1  
    fA[fD[i]] ++;          S2  
}
```

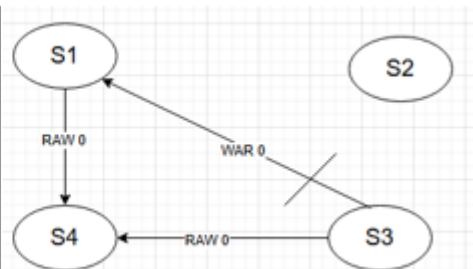
Aquest bucle és més delicat perquè fa accessos indirectes als arrays fS i fA mitjançant fD[i]. Si dues iteracions diferents tenen el mateix valor de fD[i], podrien intentar actualitzar simultàniament la mateixa posició de fS o fA. Això genera una possible condició de carrera.

Per paral·lelitzar-lo correctament, cada thread utilitza acumuladors locals fS_local i fA_local. D'aquesta manera, durant el recorregut dels 600.000 elements, cada thread només modifica les seves còpies locals. Al final de la regió paral·lela, els resultats locals s'acumulen als arrays globals dins d'una secció crítica.

Aquesta solució introdueix una sincronització final, però evita les condicions de carrera i conserva el resultat de la versió seqüencial. Com que el bucle recorre N elements, paral·lelitzar-lo sí que pot compensar el cost afegit.

2.4. Bucle 4: recalcu dels centroides

```
for(i=0;i<fK;i++)  
{  
    t = fR[i];          S1  
    if (fA[i])           S2  
        fR[i] = fS[i]/fA[i]; S3  
    dif += abs(t - fR[i]); S4  
}
```



En aquest bucle es recalculen els centroides a partir de fS i fA, i s'acumula a dif la variació respecte els centroides anteriors. La variable dif controla la condició de parada del do-while.

Teòricament es podria paral·lelitzar fent una reducció sobre dif, però el bucle només té G = 200 iteracions. Per aquest motiu, a la versió final s'ha deixat seqüencial, ja que el guany esperat és molt baix i es redueix l'overhead de sincronització.

3. Proposta de paral·lelització

La proposta final manté el main pràcticament igual que el codi seqüencial. La inicialització de V no s'ha paral·lelitzat perquè utilitza rand(), i paral·lelitzar-la podria alterar l'ordre de generació dels valors i, per tant, canviar el comportament del programa. La inicialització dels primers 200 centroides tampoc s'ha paral·lelitzat perquè el nombre d'iteracions és massa petit.

Dins de kmean, s'ha paral·lelitzat el bucle 1 amb #pragma omp parallel for, ja que cada element es pot assignar de manera independent. S'utilitza schedule(static) perquè el cost de cada iteració és molt semblant: per a cada element es comparen els 200 centroides.

El bucle 2 i el bucle 4 s'han deixat seqüencials perquè només treballen sobre 200 posicions. En proves anteriors, paral·lelitzar aquests bucles petits podia afegir overhead sense millorar de manera significativa el temps total.

El bucle 3 s'ha paral·lelitzat amb una regió parallel, acumuladors locals per thread i una secció crítica final. Aquesta estratègia evita les condicions de carrera sobre fS i fA.

Finalment, qs també s'ha deixat seqüencial, ja que ordena únicament 200 elements i el seu temps és de l'ordre de microsegons o mil·lisegons molt petits.

4. Codi de la proposta

A continuació es mostra el codi final utilitzat per a les execucions:

```
#include <stdlib.h>
#include <stdio.h>
#include <omp.h>

#define N 600000
#define G 200

long V[N];
long R[G];
int A[G];

void kmean(int fN, int fK, long fV[], long fR[], int fA[])
{
    int i, j, iter = 0;
    long dif;
    long fS[G];
    int fD[N];

    double t_b1 = 0.0;
    double t_b2 = 0.0;
    double t_b3 = 0.0;
    double t_b4 = 0.0;

    do
    {
        double t0;
        t0 = omp_get_wtime();

        #pragma omp parallel for private(j) schedule(static)
        for (i = 0; i < fN; i++)
        {
            int min = 0;
            long best_dif = labs(fV[i] - fR[0]);
```

```

    for (j = 1; j < fK; j++)
    {
        long d = labs(fV[i] - fR[j]);

        if (d < best_dif)
        {
            min = j;
            best_dif = d;
        }
    }

    fD[i] = min;
}

t_b1 += omp_get_wtime() - t0;

t0 = omp_get_wtime();

for (i = 0; i < fK; i++)
{
    fS[i] = 0;
    fA[i] = 0;
}

t_b2 += omp_get_wtime() - t0;

t0 = omp_get_wtime();

#pragma omp parallel
{
    long fS_local[G];
    int fA_local[G];

    for (int k = 0; k < fK; k++)
    {
        fS_local[k] = 0;
        fA_local[k] = 0;
    }

    #pragma omp for schedule(static)
    for (i = 0; i < fN; i++)
    {
        int c = fD[i];

        fS_local[c] += fV[i];
        fA_local[c]++;
    }

    #pragma omp critical
    {
        for (int k = 0; k < fK; k++)
        {
            fS[k] += fS_local[k];
            fA[k] += fA_local[k];
        }
    }
}

t_b3 += omp_get_wtime() - t0;

t0 = omp_get_wtime();

dif = 0;

for (i = 0; i < fK; i++)
{
    long old = fR[i];

    if (fA[i])

```

```

        {
            fR[i] = fS[i] / fA[i];
        }

        dif += labs(old - fR[i]);
    }

    t_b4 += omp_get_wtime() - t0;

    iter++;

} while (dif);

printf("iter %d\n", iter);
printf("Tiempo bucle1: %f\n", t_b1);
printf("Tiempo bucle2: %f\n", t_b2);
printf("Tiempo bucle3: %f\n", t_b3);
printf("Tiempo bucle4: %f\n", t_b4);
}

void qs(int ii, int fi, long fV[], int fA[])
{
    int i, f;
    long pi, pa, vtmp, vta, vfi, vfa;

    pi = fV[ii];
    pa = fA[ii];

    i = ii + 1;
    f = fi;

    vtmp = fV[i];
    vta = fA[i];

    while (i <= f)
    {
        if (vtmp < pi)
        {
            fV[i - 1] = vtmp;
            fA[i - 1] = vta;

            i++;

            vtmp = fV[i];
            vta = fA[i];
        }
        else
        {
            vfi = fV[f];
            vfa = fA[f];

            fV[f] = vtmp;
            fA[f] = vta;

            f--;

            vtmp = vfi;
            vta = vfa;
        }
    }

    fV[i - 1] = pi;
    fA[i - 1] = pa;

    if (ii < f)
    {
        qs(ii, f, fV, fA);
    }
}

```

```

    if (i < fi)
    {
        qs(i, fi, fV, fA);
    }
}

int main()
{
    int i;
    double tiempo_qs = 0.0;

    printf("Threads: %d\n", omp_get_max_threads());

    for (i = 0; i < N; i++)
    {
        V[i] = (rand() % rand()) / N;
    }

    for (i = 0; i < G; i++)
    {
        R[i] = V[i];
    }

    kmean(N, G, V, R, A);

    double t0 = omp_get_wtime();

    qs(0, G - 1, R, A);

    tiempo_qs = omp_get_wtime() - t0;

    printf("Tiempo del quickSearch: %f\n", tiempo_qs);

    for (i = 0; i < G; i++)
    {
        printf("R[%d] : %ld te %d agrupats\n", i, R[i], A[i]);
    }

    return 0;
}

```

5. Execució i metodologia de mesura

Les execucions s'han fet amb gcc -O3 -fopenmp. En els scripts d'execució s'ha fixat OMP_NUM_THREADS al nombre de threads de cada prova, i també s'han utilitzat OMP_PLACES=cores i OMP_PROC_BIND=close per mantenir una afinitat més estable entre threads i cores.

A Orca s'han mesurat les configuracions de 1, 2, 4, 8, 16, 32, 64 i 128 threads. A Teen s'han mesurat les configuracions de 1, 2, 4, 8 i 16 threads.

En les proves amb pocs threads es va observar que, sense controlar l'ús de hyperthreading, el sistema podia assignar threads a CPUs lògics del mateix core físic. Per aquest motiu, en les configuracions que no necessiten SMT s'ha utilitzat --hint=nomultithread per forçar cores físics diferents.

Per calcular el speedup experimental s'ha utilitzat com a base el temps de la primera execució amb 1 thread de la mateixa màquina. Això dona una comparació coherent entre la versió OpenMP executada amb 1 thread i les execucions amb més threads.

6. Resultats

La fórmula utilitzada per al speedup és:

$$\text{Speedup} = T_1 / T_p$$

on T_1 és el temps de la versió paral·lela executada amb 1 thread en la mateixa màquina, i T_p és el temps amb p threads.

La mitjana harmònica s'ha calculat sobre les configuracions paral·leles, és a dir, exclouent el cas d'1 thread.

6.1. Resultats a Orca

Threads	Temps (s)	Speedup
1	17.10	1.00
2	8.59	1.99
4	4.34	3.94
8	2.22	7.70
16	1.15	14.87
32	0.61	28.03
64	0.36	47.50
128	0.32	53.44

La mitjana harmònica del speedup a Orca és 6.805. Per tant, es supera el mínim demanat de 6.

6.2. Resultats a Teen

Threads	Temps (s)	Speedup
1	8.27	1.00
2	4.14	2.00
4	2.08	3.98
8	1.07	7.73
16	0.96	8.61

La mitjana harmònica del speedup a Teen és 4.010. Per tant, també es supera el mínim demanat de 4.

6.3. Gràfiques dels resultats

Per complementar les taules de temps i speedup, s'inclouen a continuació diverses gràfiques de temps d'execució, speedup i eficiència per a Orca i Teen. Aquestes figures permeten visualitzar de forma més clara l'escalabilitat de la proposta.

Gràfiques d'Orca

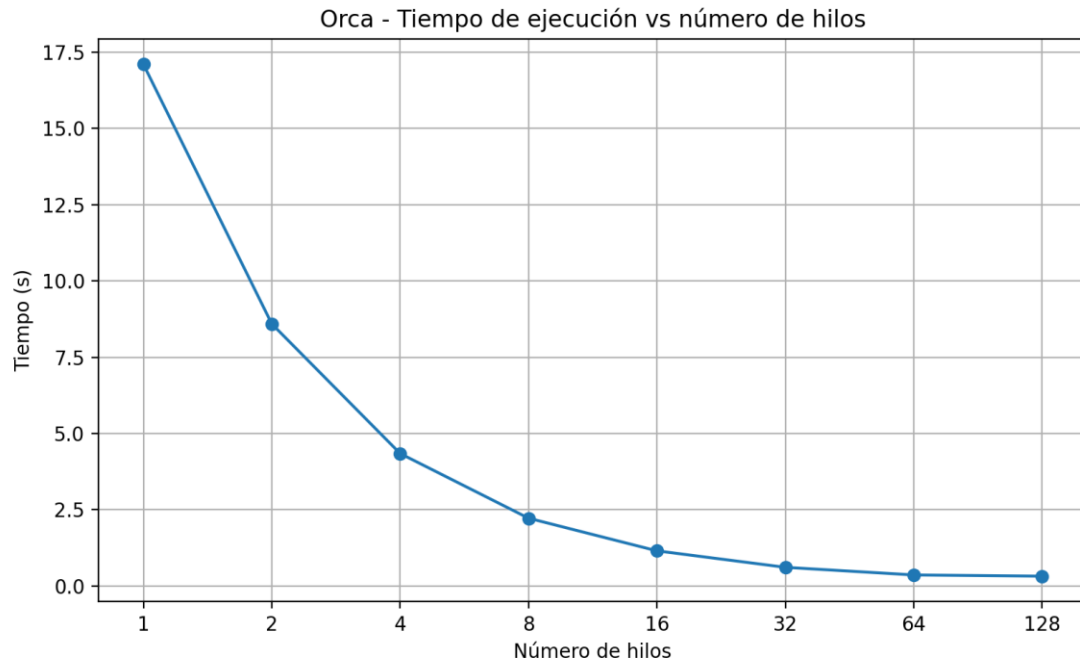


Figura 1. Temps d'execució a Orca en funció del nombre de threads.

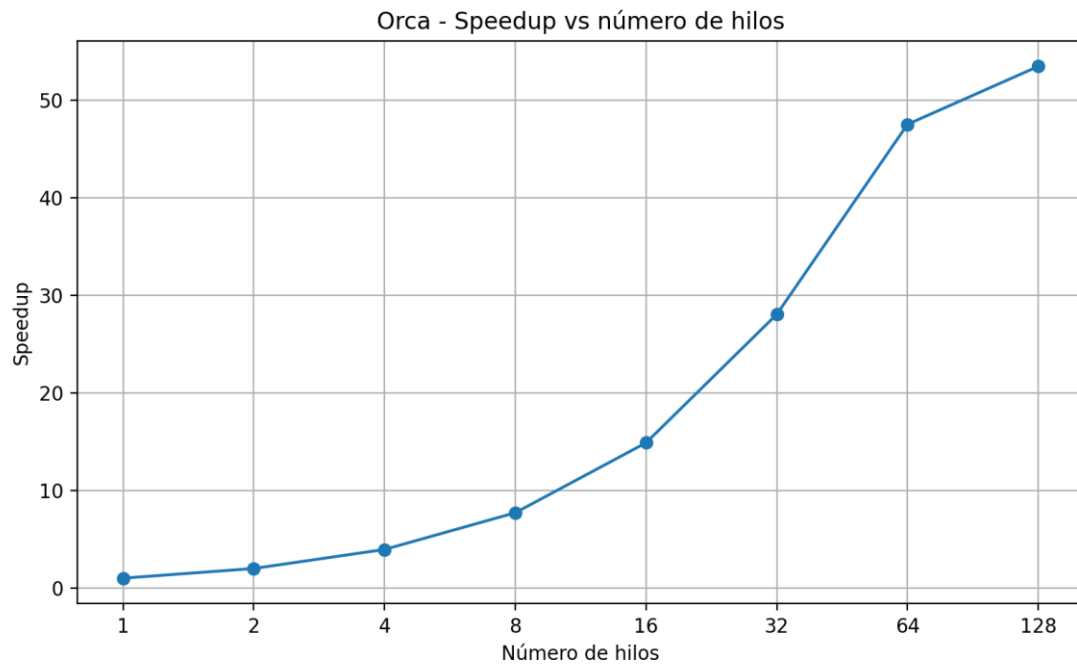


Figura 2. Speedup a Orca en funció del nombre de threads.

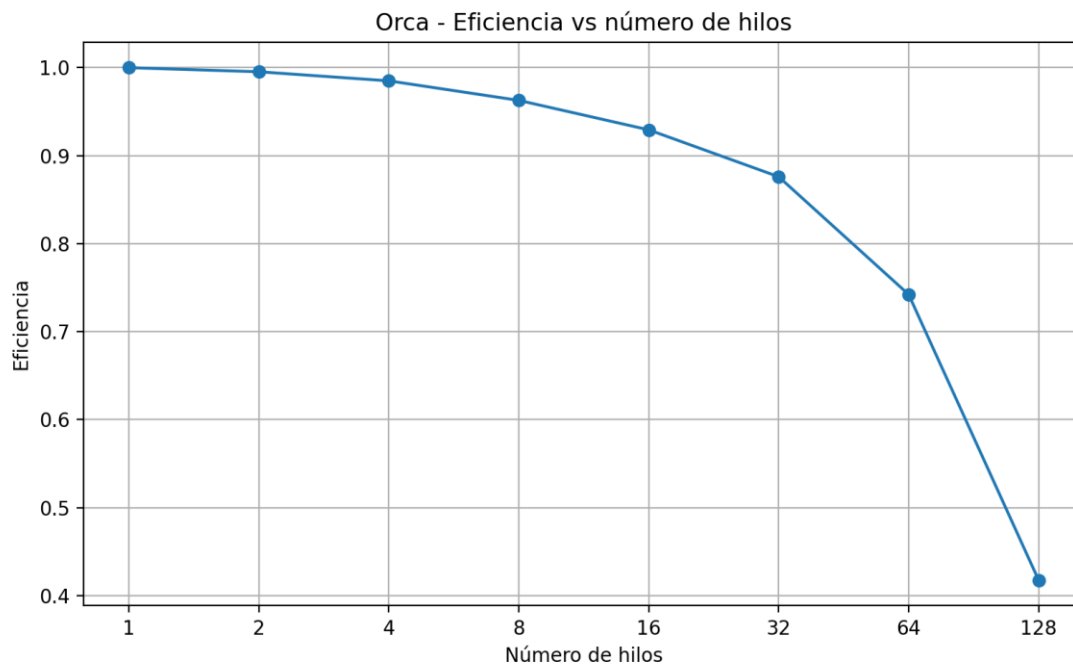


Figura 3. Eficiència a Orca en funció del nombre de threads.

A Orca s'observa una reducció molt marcada del temps d'execució a mesura que augmenta el nombre de threads. El speedup creix de manera clara i la mitjana harmònica obtinguda és de 6.805, superior al mínim exigít. L'eficiència decreix progressivament amb el nombre de threads, cosa esperable per l'augment de l'overhead i de la sincronització, però es manté prou bona per complir l'objectiu de la pràctica.

Gràfiques de Teen

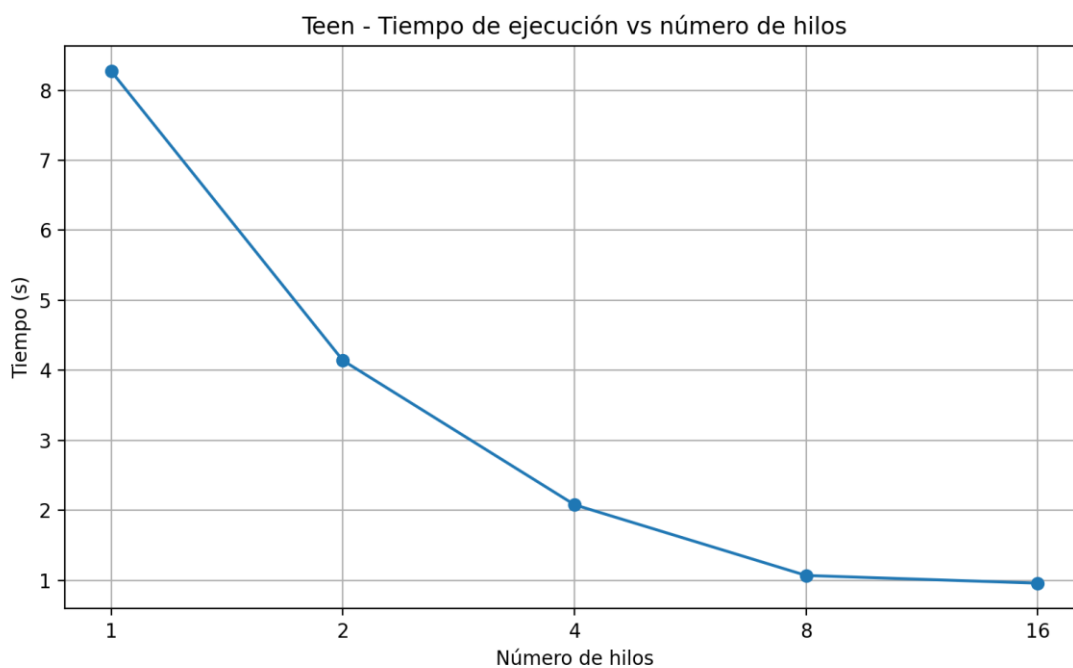


Figura 4. Temps d'execució a Teen en funció del nombre de threads.

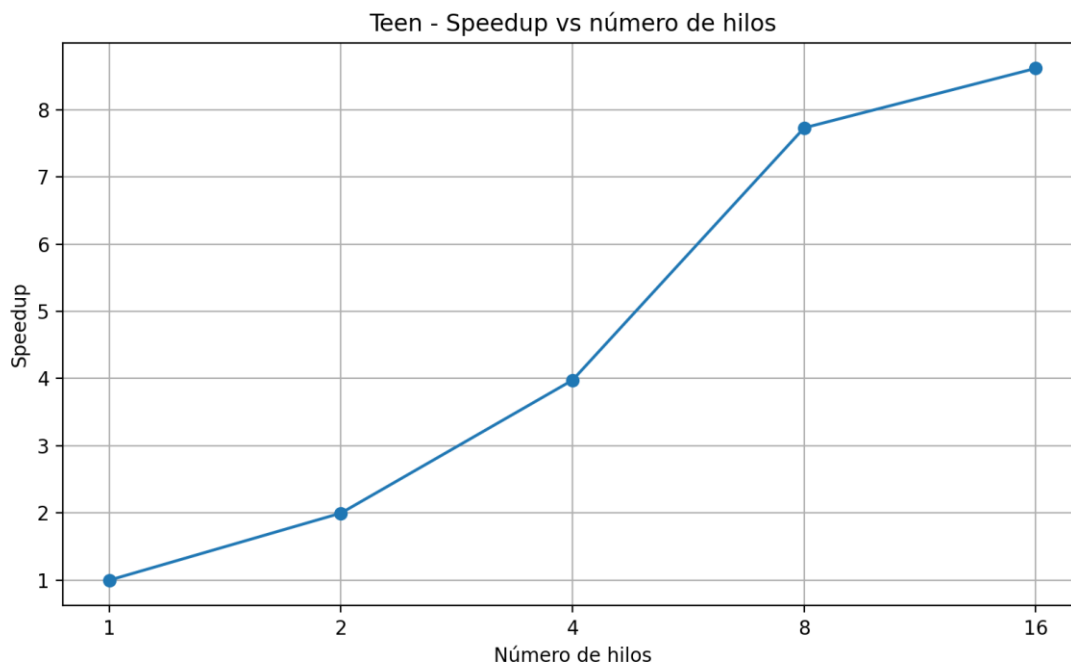


Figura 5. Speedup a Teen en funció del nombre de threads.

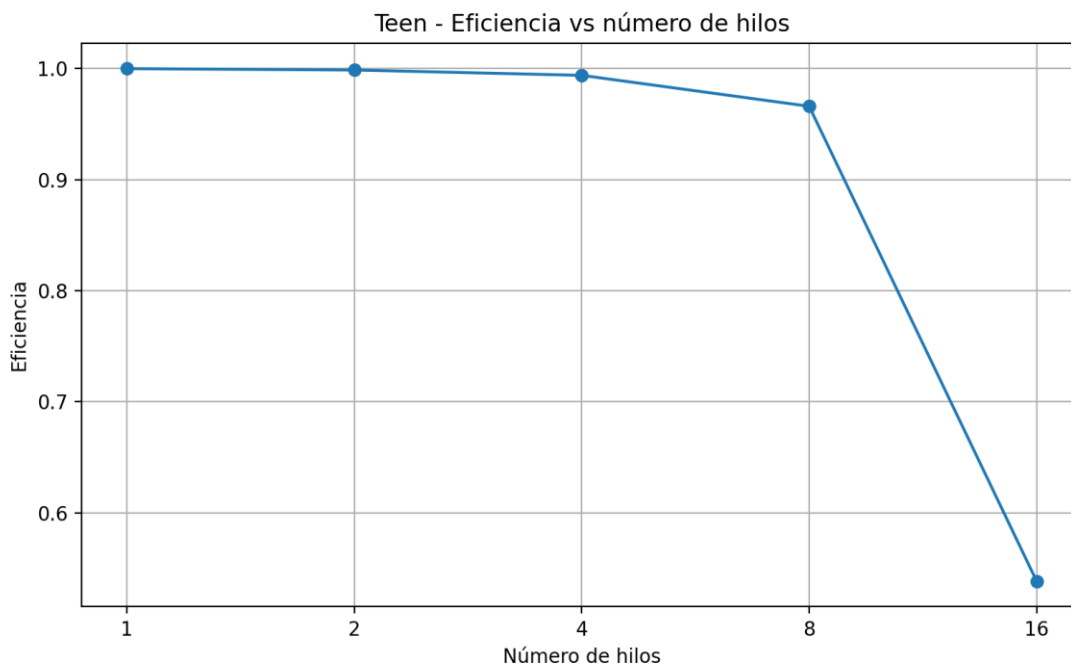


Figura 6. Eficiència a Teen en funció del nombre de threads.

A Teen també es veu una millora clara del rendiment en augmentar el nombre de threads. La mitjana harmònica del speedup és 4.010, per tant supera lleugerament el llindar mínim demanat. Igual que a Orca, l'eficiència va disminuint quan es fan servir més threads, però la tendència global continua sent positiva i mostra que la paral·lelització és útil i escalable.

7. Validació de la sortida

Per comprovar que la paral·lelització és correcta, s'ha comparat la sortida de la versió paral·lela amb la sortida de la versió seqüencial. En totes les execucions comprovades, el nombre d'iteracions ha estat el mateix: iter 147.

També coincideixen els 200 valors finals $R[i]$ i el nombre d'elements agrupats $A[i]$. Per tant, la paral·lelització conserva el resultat funcional del programa seqüencial.

Aquesta comprovació és important perquè el bucle 3 modifica l'ordre intern en què s'acumulen els valors. Tot i això, com que es treballa amb enters long i cada thread acumula localment abans de combinar els resultats, la sortida final coincideix amb la seqüencial.

8. Conclusions

La part dominant del programa és el bucle 1 de kmean, amb un cost $O(N \cdot G)$. La seva paral·lelització és la que aporta la major part de la millora de rendiment.

El bucle 3 també s'ha paral·lelitzat, però requereix una estratègia específica amb acumuladors locals per evitar condicions de carrera sobre fS i fA . La secció crítica final introdueix overhead, però el cost del bucle és prou gran perquè la paral·lelització compensi.

Els bucles petits, com la inicialització dels acumuladors i el recalcul dels centroides, s'han deixat seqüencials perquè només recorren 200 elements. En aquest cas, reduir l'overhead és més important que paral·lelitzar totes les parts del programa.

Els resultats finals són satisfactoris: la mitjana harmònica del speedup és 6.805 a Orca i 4.010 a Teen. Això compleix els mínims demanats per l'enunciat.

Finalment, s'ha comprovat que la versió paral·lela dona la mateixa sortida que la seqüencial, de manera que la paral·lelització es pot considerar correcta tant funcionalment com en termes de rendiment.